

Run-Time Shape Padding Is Nearly Free; Coverage Is Paid at Boot: Compiled-Shape Quantization in TPU Serving, with a GPU Control

Stack: vLLM 0.25.0 and tpu-inference 0.25.0 on a v5litepod-4 TPU slice (JAX 0.10.2, tensor-parallel degree TP=4), with an NVIDIA L4 for the GPU control.

Abstract

Accelerator serving stacks round every step up to a compiled or captured shape, on the assumption that rounding up means paying for the shape rounded up to — the premise behind length bucketing, shape-aware admission control, and ladder design. We measure what is actually paid, on one TPU slice and one GPU (stack above), and **find it false there on two of the three dimensions shape coverage quantizes.**

Per-request prompt-length padding **does not exist**. Request-slot padding is **free** — under 0.7 μ s per padded slot against 27.5 μ s if paid, because the attention kernel does no work for unused slots — and a GPU control replicates this at 67 μ s (0.6% of the step) on the one dimension both architectures share; the token dimension is TPU-only. Token padding is **the exception**: real arithmetic, paid at 23.1% of nominal at batch 4 and indistinguishable from zero at batch 16.

On that dimension, ladder design still pays, and the effective variable is placement, not shape count. At low batch, below saturation, and without prefix caching, one entry added where the workload needs it cuts latency 12.1% at the straddling prompt length, at no memory cost, while a uniformly finer ladder buys nothing further and fails to boot near the stock memory fraction. Chosen offline from a length distribution, a placement predicts that gain to within 6%. The gain is a latency reduction below saturation (46% at the knee), not added capacity (2.6% at saturation), and shrinks from 12.3% to 1.7% once prefix caching is on — the regime it pays in is narrow and named here rather than reached three appendices later.

What shape coverage costs is not run-time padding but a boot-time charge: compilation and capture time, which a persistent cache largely amortizes, and compilation memory headroom, which it does not — though the ladder recommended here boots at the stock

fraction and pays neither. Every number here regenerates from captured data by a script that fails on disagreement; the mechanisms proposed to explain them receive no equivalent check.

1 Introduction

A GPU serving stack resolves kernel shapes at runtime; a TPU stack cannot. XLA, the compiler that turns a TPU model into fixed-shape executables, compiles for fixed shapes, and recompiling per request is impossible at serving latencies, so vLLM’s TPU backend precompiles a ladder of shapes and rounds every step up to one of them. vLLM’s CUDA path does something structurally similar for a different reason: it captures one CUDA graph per batch size from a fixed set and pads a batch up to the next captured size.

Both designs create the same apparent inefficiency, and a literature has grown around removing it. **BucketServe** derives an optimal length-bucket boundary and then declines to compute it, describing it as *"computationally expensive to calculate in practice."* **LAPS** captures a graph per (length, batch) cell and notes that *"the number of graphs must be limited."* Both take padding paid at run time for granted.

This paper instruments the three quantized dimensions of a TPU serving stack, isolates the mechanism responsible for each, and repeats the central measurement on a GPU using the same framework and instrument. Run-time padding proves close to free on both, and the cost of shape coverage proves to be a boot-time charge that neither system measures.

A second theme shaped what was measured: our measurements have proved durable and our explanations have not, with four of the last five retractions being claims about mechanism rather than a reported number. §2 states the protocol adopted in response; §6 reports what that asymmetry does and does not license.

Contributions.

1. **The premise, measured** (§4.1, §4.2, §4.5). Padding a batch up to a compiled or captured entry is close to free on a TPU ladder and on CUDA-graph capture alike; what shape coverage costs is paid at boot, not per step. The GPU comparison covers the request dimension, which is the dimension graph capture quantizes.
2. **The request ladder reported by the TPU stack differs from the one its attention kernel executes** (§4.1) — confirmed from the source code, a paired hardware experiment, and the compiler-emitted kernel name, and absent from the RPA (Ragged Paged Attention) paper, from LENS, and from vendor documentation.

3. **A mechanism for free request padding** (§4.1): the ragged kernel performs under 0.7 μ s of work per padded slot, against 27.5 μ s if it were paid, established by reducing the compiled slot count by a factor of 32 for a -0.9% change. The memory-bandwidth explanation the same data invites is tested and rejected.
4. **Ladder design as placement rather than cardinality** (§4.3, Appendix §10.3): a fourteen-shape ladder placed against the workload reduces end-to-end latency by 12.1% at no memory cost, while a uniformly finer ladder achieves the same reduction and additionally costs 8.8% of cache capacity and 53% more startup. The choice can be made offline from a length distribution, predicting the measured result to within 6%.
5. **When ladder placement pays, and when it does not** (Appendix §10.1–§10.3): a latency gain below saturation rather than added capacity — median latency falls 46% just below the knee, sustained throughput rises 2.6% at it — and one that shrinks from 12.3% to 1.7% once prefix caching (vLLM’s default) moves the prefill onto a different compiled entry, so entries must target uncached prefill lengths.
6. **A test of the measurements by construction** (§5): four optimizations designed against the premise and rejected, their failures matching what the measurements predict — those aimed at the request dimension or per-request length padding cannot pay, since §4.1 and §4.2 find no cost there to recover.

Scope. This paper carries two separable contributions, not one continuous argument: (a) a falsification of the runtime-padding-cost premise on two of the three dimensions (§4.1, §4.2, §4.5) — the well-supported half, resting on discriminating experiments rather than fits; and (b) a placement heuristic for the one dimension where the premise survives (§4.3, §4.4) — useful and already deployable, but a static offline recipe rather than an adaptive system, and weaker evidence than (a). It is primarily a measurement study; its one intervention is set through a documented environment variable rather than a patch, so the measurement does not depend on a modified scheduler’s correctness.

2 Method

Controlled variables. Chunked prefill is **enabled** throughout — the stack’s default and the configuration production runs; §4.2 reports a `-no-enable-chunked-prefill` control for the one result where it could be confounding. This matters for how the ladder results should be read: under chunked prefill the scheduler selects a compiled shape for the

packed step, not for an individual request, so a statement that a 1200-token prompt pads to 2048 describes what executes only while that prompt prefills alone in its step. At the two concurrent requests of §4.3 it does, and the evidence is internal to the measurement — two cells saving 512 and 1024 padded tokens agree on cost per padded token to within 1%, which co-scheduling would not produce, since a packed step would save a different amount in each cell. Appendix §10.1 measures where that ceases to hold and Appendix §10.3 restores it deliberately by staggering arrivals.

Prefix caching is disabled and asserted, except in Appendix §10.2 where it is the treatment. Chunked prefill, `max_model_len`, `max_num_batched_tokens`, tensor-parallel size, `gpu_memory_utilization`, `XLA_FLAGS` and `ATTN_BUCKETIZED_NUM_REQS` are recorded. Every run’s own configuration is checked against the fixed set of required values before any hardware work starts, and aborts if a controlled variable is missing or undeclared-and-wrong — three runs rejected for a drifted declared value, one for an undeclared deliberate variation. That check reads the configuration as *declared*. A second one now reads it as *executed*: each run scrapes the live server’s own `/v1/models` and aborts if what is being served contradicts what the run declared. It is narrow — `max_model_len` is the only controlled variable that endpoint reports — and it is new, added after Appendix §10.12 demonstrated the hole by falling into it (declared 8192, served 4096, undetected, costing that run a boundary). An unreachable endpoint is reported as *unverified* rather than passing, so that "nothing disagreed" and "nothing was compared" stay distinguishable.

Units and instrument scope. All timings are measured server-side, as differences between Prometheus histogram snapshots taken before and after each measurement block; client-side wall-clock timings are not used, since they include round-trip time, HTTP overhead, tokenization and queueing delay. A step-scoped property requires a step-scoped instrument: single-step execution is verified per dispatch from the count delta of `iteration_tokens_total`, and dispatches that split are excluded rather than averaged. §6 notes that errors of this kind — an analysis that measures something other than its target — are the one class no check described here covers.

Request arrival. Where a measurement requires n requests to reach the scheduler within one step, they are released from a thread barrier after every connection is established, so arrival spread is measured in microseconds rather than milliseconds. §4.2 shows that this changes what is measurable.

Definitions. Three quantities are used throughout.

- **Flatness** — how far a short request’s cost sits from what proportional scaling would predict,

as a fraction of the distance to the full-bucket cost: $(\text{cost}(L) - p) / (\text{cost}(B) - p)$, where $p = \text{cost}(B) \cdot L/B$. A value of 1.0 is a pure staircase, meaning cost is independent of true length and padding is fully paid; 0.0 is pure linearity, meaning cost is proportional to real tokens and padding is free.

- **Share of nominal padding paid** — $(\text{measured} - \text{real}) / (\text{padded} - \text{real})$, where **real** is the cost ratio predicted by real tokens alone and **padded** the ratio predicted if the full compiled shape were paid. Zero means padding is free; one means it is fully paid, which is what the compiled-shape premise predicts.
- **Model rejection, quoted as a percentage** — the amount by which a candidate model’s prediction exceeds the measurement, as a fraction of the prediction.

Registered predictions. A proposed mechanism must state a falsifiable number before the measurement that tests it, fixed in the run’s configuration file before any result is seen — a mechanism stated only in prose cannot be rejected by any check in this pipeline, whereas a number can. Three such predictions failed during this work, each producing a result its confirmation would not have: the sharding ablation yielded a two-term cost model bounding per-step fixed overhead (Appendix §10.4), the model-scale ablation found the regime map set by batch size and dtype rather than parameter count (Appendix §10.4), and the concurrency sweep found padding migrating from individual requests to the packed step under chunked prefill (Appendix §10.1).

Statistics. Medians over repeats, with 95% confidence intervals from 10,000 bootstrap resamples. Intervals are reported wherever a claim depends on the size of a difference, because §4.4 establishes that some cells are far noisier than others.

Models. Qwen3-4B is the primary model; SmolLM2-1.7B (head dimension 64, multi-head attention) and TinyLlama-1.1B (head dimension 64, grouped-query attention at 8:1) are used for the architecture contrast.

Traceability. Every run writes `meta.json` before doing work, records the configuration hash, git SHA and dirty flag, appends to a manifest, and is never overwritten. Numerical claims are tied to run identifiers and recomputed from captured data by `scripts/paper_numbers.py`; `./reproduce_all.sh` regenerates every number and figure and exits non-zero if any disagrees.

Table 1: The three quantized dimensions of the TPU serving stack, read from source.

	quantizes	ladder
D1	prompt length → prefill shape	<i>does not exist</i>
D2	scheduled tokens / step	[16, 32, ..., 8192]
D3	requests / step	[8, ..., 256] non-attention; [256] attention

3 Three quantized dimensions

A serving step is rounded up to a compiled shape in three independent ways, and this paper keeps them separate because each has a different mechanism and a different cost. The ladders below are those the TPU backend constructs at startup and prints in its own warmup log, so they can be read off a running server rather than inferred:

Keeping them separate matters because they behave differently: request-slot padding turns out to be free, per-request length padding does not exist, and only token padding is paid. A measurement of one therefore says nothing about the others, and every result below states which dimension it concerns.

These ladders are properties of the scheduler rather than of the model: `get_token_paddings`, which builds the D2 ladder, takes a minimum size, a maximum size, and a gap, with no reference to the model, and D3 is built the same way. A mixture-of-experts model therefore gets the same ladder as a dense one, and no fourth quantized dimension exists for it either — expert-side padding there is a static weight layout fixed at load, not a shape the scheduler selects. Read directly from the implementation rather than measured, since which shapes exist is a fact about the code: architecture-generalizability here is a question about cost per padded token, never about which shapes an architecture is padded to.

4 Results

4.1 Request-dimension padding is free, and the mechanism is the data structure

`envs.ATTN_BUCKETIZED_NUM_REQS` defaults to `False`, and when it is off, `get_attn_req_paddings` returns `[max_req_size]`, a single bucket. Every boot prints both ladders, and they disagree:

```
Prepared request paddings: [8, 16, 32, 64, 128, 256]
Prepared attn request paddings: [256]
```

Executed request-slot count The attention kernel therefore executes at a fixed size of 256 request slots,

independently of the batch size. Three measurements establish that this padding carries negligible cost.

The first is a paired experiment. Enabling the ladder for attention, so that it matches the one the stack advertises, changes decode by 0.0%: the two arms are identical to 0.1 ms at $n=8$ and $n=9$. This comparison excludes a large effect and supports nothing finer, since the paired difference is $+0.00$ ms with a 95% bootstrap interval of $[-10.7, +10.8]$ ms at $n=8$, or $\pm 20\%$ of the decode phase.

Second, the compiler records the padding in the name it gives the kernel. The decode attention kernel is emitted under a name that encodes the shapes it was compiled for, and the field holding the request-slot count reads 256 regardless of the batch actually being served. The stack therefore reports, in its own compilation output, that attention runs at 256 slots.

The third is an operator-level measurement, and by itself it cannot discriminate between the hypotheses either. With prompt and output length fixed, attention device time over a full 64-step generation rises from 16,830 μ s at $n=1$ to 163,535 μ s at $n=16$ — not flat, but flatness was never the alternative, since padded slots hold no key-value blocks and no kernel could do work proportional to state that does not exist. The only cost padding could plausibly carry is a fixed per-slot overhead — walking 256 block-table entries, loading 256 metadata records — which is constant in n by construction. Fitting $T = a \cdot n + b$ gives a fixed term of 7,050 μ s (residuals under 2.3%), 42% of attention time at $n=1$ falling to 4% at $n=16$; were that a 256-slot padding cost it would be 27.5 μ s per slot. The fit is equally consistent with padded slots being skipped and with a fixed cost paid for all 256, and so establishes neither.

Discriminating experiment: reducing the slot count

The discriminating experiment reduces the compiled slot count and observes the fixed term. Enabling `ATTN_BUCKETIZED_NUM_REQS` compiles attention at 8 slots rather than 256, a 32-fold reduction, profiled in both arms at the same real batch size: device time moves by -0.9% at $n=1$ (16,898 $\rightarrow$ 16,749 μ s), -0.5% at $n=2$ (26,130 $\rightarrow$ 25,999 μ s), and -0.0% at $n=8$ (85,081 $\rightarrow$ 85,055 μ s), and the fitted fixed term b moves by -2% (7,158 $\rightarrow$ 6,991 μ s).

A per-padded-slot cost would be expected to fall by approximately 97% when the slot count is reduced by a factor of 32. The measured reduction is 2%. The fixed term therefore represents block-table and dispatch overhead rather than padding. Stated as the bound the experiment supports: removing 248 of 256 slots moves the fitted fixed term by 167 μ s, so a padded request slot costs **under 0.7 μ s**, against the 27.5 μ s per slot that fully-paid padding would imply. This is a factor of 41, and it is a bound rather than zero: at 256 slots the residue is up to approximately 170 μ s, about 1% of attention time

Table 2: Share of nominal token padding paid, by batch size. Rows are confounded: each was measured over a different set of ladder boundaries.

batch size	median	mean	95% CI
1-2	~85%	—	<i>not computed</i>
4	23.1%	20.2%	[13.5%, 24.4%]
8	14.3%	11.8%	[0.2%, 21.0%]
16	-2.7%	-5.9%	[-15.4%, +0.5%]

at $n=1$.

This measurement also runs the high-power version of the paired experiment above: that test used $n=8$ and $n=9$, where a fixed 256-slot cost would have been about 8% of attention time, against 42% at $n=1$ — and at $n=1$ the measured difference is -0.9%, so the 0.0% result was not a low-power test concealing an effect, since the effect is absent where it would have been largest.

The memory-bandwidth account — that free padding follows from the step reading the whole weight set regardless of batch — is tested and rejected using measured chip utilization and memory-bandwidth utilization; Appendix §10.7 has the account and the measurements against it. No promotion cost exists at the 8 $\rightarrow$ 16 request edge under the default configuration either, because the attention kernel is already compiled at 256 slots and does not distinguish the two batch sizes.

4.2 Token-dimension padding is paid, and its share falls with batch size

Token padding is a different quantity with a different mechanism. A prefill step carries hundreds to thousands of tokens, and padded tokens are real floating-point operations, because the kernel computes them. Token padding is therefore paid wherever the step’s cost is dominated by arithmetic on those tokens.

Table 2 reports the share of nominal padding paid at a compiled boundary, with batch size fixed and sequence length held near-constant:

Confounding across ladder boundaries These rows are confounded — over 1, 4, 3 and 3 boundaries respectively (9–15 clean dispatches/arm at $n=4$, 3–5 at $n=8$, 7–11 at $n=16$), because each was measured over a different set of ladder boundaries. At fixed $n=4$ the paid share rises with boundary size, from 10.0% at 512 $\rightarrow$ 1024 to 24.8% at 4096 $\rightarrow$ 8192, so the particular boundaries a row contains shift its value independently of batch size. The sets differ: $n=8$ lacks 512 $\rightarrow$ 1024, the lowest-paying boundary, and $n=16$ lacks 4096 $\rightarrow$ 8192, the highest. Both omissions push in the same direction and exaggerate the decline. Comparing rows measured over different boundary sets is the most common error in this work, and it occurred here in a headline table.

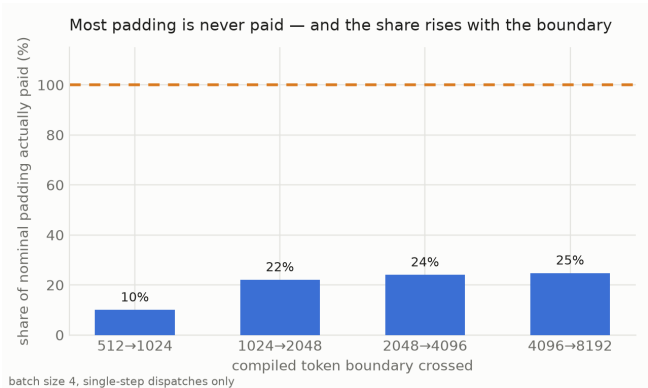


Figure 1: Share of nominal padding actually paid at each compiled boundary, against the 100% the compiled-shape premise predicts.

Restricted to the two boundaries present in every row (1024→2048 and 2048→4096), the decline remains: mean paid share is 23.1% at $n=4$, 10.6% at $n=8$ (down from 14.3% unmatched), and -1.1% at $n=16$. An interval bootstrapped over two boundaries would not be meaningful, so this establishes the ordering of the rows rather than their absolute values. The defensible statement is ordinal rather than monotone: the paid share is substantial at $n \leq 2$, intermediate at $n=4-8$, and indistinguishable from zero at $n=16$, with only the $n=4$ against $n=16$ contrast separated at interval level. The $n \leq 2$ row itself rests on a single boundary, not one of the matched pair, and carries no interval — simultaneously the largest figure in the paper and the least well supported. The $n=8$ value of 14.3% excludes split dispatches; including them pools partial steps into the median and yields 16%, and the exclusion is the correct treatment.

Measuring $n \geq 16$ required fixing the request launcher first: a naive one makes the quantity appear unmeasurable above $n=8$ by racing every dispatch into a split step. Appendix §10.8 has the launcher fix and what it recovers.

Paid share against boundary size The share paid rises with the size of the boundary. At fixed $n=4$ it is 10.0% at 512→1024, 22.1% at 1024→2048, 24.0% at 2048→4096 and 24.8% at 4096→8192. Batch size is held constant across these figures, so the comparison is between boundaries and not between load points.

Per-request length padding Per-request length padding does not exist. Holding batch size and total tokens fixed and varying only the spread of request lengths, the batch-padding model is rejected in every ragged cell by at least 44%, and cost tracks packed tokens instead. The rejection range extends to 618%, but a span of more than an order of magnitude is a directional refutation rather than a measured effect size,

so the minimum is the figure that carries the claim. Uniform controls, in which all candidate models agree, match to 1.9%. This is not an artifact of chunked prefill: with `-no-enable-chunked-prefill` the result is unchanged, the packed model winning 8 of 10 ragged cells and batch padding being rejected by 75–579%.

How padded share varies with workload shape, not just batch size, is a separate question from the one this section answers; Appendix §10.9 sweeps four length distributions at matched offered tokens and reports a retracted explanation along the way. A single recoverable-headroom figure — multiplying the padded share above by the paid share reported earlier in this section — would be invalid: paid share moves with batch size, so such a figure would describe no configuration that was ever run, which is why the two factors are reported separately.

4.3 Ladder design is placement, not cardinality

Token padding is the one dimension on which the premise survives, so it is the dimension an intervention should target. `VLLM_TPU_BUCKET_PADDING_GAP` provides the lever: unset, the TPU backend compiles ten exponentially spaced token shapes; set to 512, it compiles twenty-one linearly spaced ones. No patch is required.

To separate the effect of the ladder from differences between server instances, the design includes control prompt lengths. Of four prompt lengths, two pad to the same compiled shape on both ladders and two pad to a smaller shape on the fine ladder only, so the placebo cells measure everything that differs between two server boots except padding. Qwen3-4B, v5litepod-4, TP=4, two concurrent requests, `output_len=32`, 18 repeats per cell pooled over both arm orders:

The two treated cells agree on cost per padded token to within 1%, at 34.9 and 35.3 μ s, while the two placebo cells differ by +0.5 ms on average (+0.8 and +0.1 ms) — the principal evidence for the effect, since an arm-level offset (one server instance running uniformly slower) would produce a constant difference between the arms and therefore a per-token figure inversely proportional to tokens saved, whereas a padding effect scales with tokens saved, which is what the measurements show. For scale, a real prefill token costs 46.6 μ s on the same arm (1200→3000 tokens for 83.9 ms), so at two concurrent requests a padded token costs about three quarters of a real one; expressed as percentages, the twenty-one-shape ladder reduces end-to-end latency by 8.7% at prompt 1200 and 12.5% at prompt 3000.

Cost of the finer ladder The cost of the finer ladder is startup and memory headroom, not cache capacity. Measured with the compiled-shape cache cleared before each boot:

Table 3: Ten-shape against twenty-one-shape ladder at two concurrent requests. Prompts 300 and 600 pad identically on both ladders and act as controls. Intervals are within-boot; the between-boot floor Table 9 measures is 4-11 ms, wider than any interval shown.

prompt	pads to (10 shapes)	pads to (21 shapes)	tokens saved	10-shape	21-shape	difference	[95% CI]
300	512	512	0	149.8 ms	150.6 ms	+0.8 ms	[+0.1, +1.6]
600	1024	1024	0	168.5 ms	168.6 ms	+0.1 ms	[-0.5, +0.6]
1200	2048	1536	512	206.0 ms	188.2 ms	-17.9 ms	[-18.2, -17.5]
3000	4096	3072	1024	289.9 ms	253.7 ms	-36.2 ms	[-36.5, -36.0]

Table 4: What the finer ladder costs: startup and the memory fraction at which the server will start; key-value capacity at each fraction follows in text.

	10 shapes	21 shapes
cold warmup	285 s	436 s (+53%)
compiled-shape cache on disk	43 MB	92 MB
highest memory fraction that boots	0.92 (the default)	0.85

The warmup figures are cold. A persistent compilation cache amortizes them across restarts, and warm boots of the same two ladders measured 165–315 s, reflecting cache reuse rather than ladder size.

At a memory fraction of 0.85 the two ladders report the same 335,104 tokens, to the token. That equality also bounds any residual effect, since capacity is block-quantized and the 49 MB difference in executables would amount to roughly 1,300 tokens, or 81 blocks, and would have been visible. What the finer ladder costs is the backoff it forces: the ten-shape ladder runs at the 0.92 default for 367,360 tokens, the twenty-one-shape ladder does not start above 0.85, and the difference between those operating points is 32,256 tokens, or 8.8% of capacity.

That the mechanism is a compilation-headroom cliff rather than steady-state cache competition was established by tracing the failure directly rather than assuming it; Appendix §10.10 has the `RESOURCE_EXHAUSTED` forensics.

Separating placement from cardinality The benefit, however, is bought by a different variable than the cost. The padding-gap lever changes the spacing law and the shape count together, so the measurements above attribute a benefit and a price to the same parameter without separating them. Every cost scales with cardinality: warmup, resident executables, and the headroom that sets the boot cliff. Two intermediate gaps separate them (Table 5). The default, gap-1024 and gap-2048 rows are measured together, at 0.92, in one server boot; the gap-512 row is not remeasured here and repeats the twenty-one-shape values already established in Table 3, from a different boot the day before:

† Reused from Table 3’s boot, not this one. That boot’s own default arm read 206.0 ms at prompt 1200 and 289.9 ms at prompt 3000 — 9.4 ms and 7.3 ms below this boot’s default row for a nominally identical configuration, in line with the 4–11 ms between-instance offsets

Table 9 measures directly. Any comparison that mixes the gap-512 row with a value from this table’s own boot carries that much additional, uncorrected uncertainty; the -12.1% figure below does not, since default and gap-1024 are both this boot’s own measurements.

Against the ten-shape ladder, the raw table differences are -4.5 ms at prompt 1200 and -40.7 ms at prompt 3000; both are corrected by +4.7 ms — this boot’s own prompt-300 placebo offset, not itself shown above since only prompts 1200 and 3000 appear there — giving +0.2 ms and -36.0 ms (-12.1%), both from this same boot: the fourteen-shape ladder gains exactly where it places an entry the default lacks, at 3072, and nowhere else — no entry at 1536 leaves prompt 1200 unaffected, and the eleven-shape ladder, placing nothing new near either prompt, tracks the default at both. Shape count ranges from 10 to 21 while the benefit depends only on whether a boundary falls between the prompt and the next default entry.

Feasibility at the default memory fraction The fourteen-shape ladder boots at the stock 0.92 with 367,360 tokens, the same capacity as the default, so the 8.8% is the price of the twenty-one-shape ladder rather than of the optimization: a 12.1% reduction at prompt 3000 is available at full memory, for the warmup of four additional shapes, while the twenty-one-shape ladder’s further 8.7% at prompt 1200 is what costs 8.8% of capacity and 53% more startup. The recommendation is therefore not to compile more shapes but to **compile the shapes the workload’s prompt distribution straddles**: cardinality is what the stack charges for, placement is what latency responds to, and a badly placed boundary is an executable the compiler pays to produce and the scheduler never uses.

Table 5: Separating placement from cardinality. Shape count ranges from 10 to 21 while the benefit follows only whether an entry falls between the prompt and the next default entry. Gap 512 is carried over from Table 3 rather than measured in this boot; see the note below the table. Every arm here is one boot, so no interval spans the 4-11 ms between-boot floor.

ladder	shapes	1200 pads to	3000 pads to	prompt 1200	prompt 3000
default	10	2048	4096	215.4 ms	297.2 ms
gap 1024	14	2048	3072	210.9 ms	256.5 ms
gap 512 †	21	1536	3072	188.2 ms	253.7 ms
gap 2048	11	2048	4096	214.1 ms	296.1 ms

4.4 A ladder can be chosen offline from a length distribution

§4.3’s placement was chosen by knowing two prompt lengths in advance, which is an existence proof rather than a method. A method begins from a length distribution and selects a ladder without reference to latency; what follows tests that procedure on the one workload and feasible set available here, not a general claim that it always finds the right ladder.

We sampled 120 prompts from a lognormal distribution (median 1200, $\sigma=0.9$) and replayed the same lengths against every ladder, pairing the arms on workload. Each arm’s latency was then predicted from its expected padded-token count multiplied by the 35 μ s per padded token measured in §4.3:

Predictive accuracy of the offline model The offline model predicted the measured result to within 5.3%: 7.5 ms predicted against 7.1 ms measured. Inverting it gives 33.3 μ s per padded token against the 34.9–35.3 μ s of §4.3, and the two workloads share nothing, since §4.3 used two fixed lengths straddling known entries and this a heavy-tailed mixture over the whole ladder — a constant fitted on one workload and confirmed on another supports using the model for design. Ladder design therefore does not require a hardware sweep: sample the length distribution, compute expected padding per candidate ladder, and multiply by the per-token cost.

The model is per-request; the mechanism above $n=2$ is per-step. This replay ran at concurrency 2, where each prompt prefills alone in its step (§2), so pricing padded tokens *per request* is correctly specified here — which is why the agreement means anything. Appendix §10.1 shows that specification failing above $n=4$, where the packed step selects the shape. The rule is therefore validated in the isolated-prefill regime only. That limit coincides with the regime the rule is *for*, since §4.2 finds the paid share substantial at low batch and near zero by 16 — a coincidence of regimes, not a derivation.

Constraining the objective The objective must be constrained, since the unconstrained objective is the premise this paper refutes: expected padding falls monotonically with shape count, so minimizing it alone drives

the ladder toward the finest the stack can compile, which §4.3 shows the stack cannot afford. Both finer arms failed to boot at the stock memory fraction, the 35-shape ladder more decisively (23.75 MB requested against 4.94 MB free) than the 21-shape one (32.50 MB against 12.40 MB). The feasible set here was {10, 14} and the answer was 14 — the configuration rule is to select the greatest shape density that satisfies the memory constraint, with entries placed against the length distribution.

Comparison against BucketServe’s ladder objective

BucketServe specifies a different objective for the same decision — minimize expected relative waste, $E[1 - S/U_b]$ — and derives the Lloyd-Max centroid condition for it, which their paper declines to compute, calling it expensive in practice. That condition is local; we solve their objective globally by dynamic programming over a discretized length axis, $O(K N^2)$ and milliseconds, so their objective is given a better solution than their own formulation proposes. At equal shape count the padded-token objective comes out ahead, and both ladder-design objectives beat the stock ladder by far more than they differ from each other — the disagreement between objectives is small next to the decision to design a ladder at all. The direction is what the cost model predicts: relative waste treats a 10-token overshoot on a 100-token request as equal to a 1000-token overshoot on a 10,000-token one, whereas §4.3 prices a padded token at a constant, so absolute padding is the quantity that converts to time. This is an objective-level comparison, not a reimplementing of their system, and its margin is reported in Appendix §10.14 rather than here because the four arms’ co-boot provenance is not established and the margin is smaller than the between-instance offset Table 9 measures.

Applying the rule under a moving traffic distribution — re-selection cadence, the counter that would trigger it, and a correction to how a padding gap maps to a ladder — is Appendix §10.11. We have not built that controller, and report it as the operational form of the result rather than as a contribution.

Table 6: Ladders selected offline from a lognormal length distribution, with predicted and measured latency. One boot per arm; the 4-11 ms between-boot floor is not represented in these figures.

ladder	shapes	mean padded tok/req	predicted Δ	measured e2e	boots at 0.92?
default	10	602	—	219.8 ms	yes
gap 1024	14	389	-7.5 ms	212.7 ms (-7.1)	yes
gap 512	21	—	-16.2 ms	<i>does not boot</i>	no
gap 256	35	—	—	<i>does not boot</i>	no

4.5 The request dimension is small on GPU too, within a wide bound

§4.1’s request-dimension finding was measured on TPU. Whether it holds on another architecture is testable rather than assumed: same vLLM 0.25.0, same instrument, an L4 (23 GB, TP=1) in place of the v5e:

The increments are the measurement; the levels are not. Eager execution pays a constant per-operation launch overhead, so the levels differ by roughly 9.3 ms throughout, and that constant cancels in any increment — to within 9 μ s on the 8→16 step, at 1.693 against 1.684 ms, which is the control this comparison requires: the two arms measure the same underlying work plus an offset, so their difference isolates what capture adds. On that basis, padding a batch from 8 up to the captured entry at 16 costs approximately **67 μ s**, being the 0.283 ms increment with graphs against 0.215 ms without — 0.6% of a 10.9 ms step, or 4.0% of the nominal padding implied by rounding 9 up to 16.

This tests one of the three dimensions, and it is the one least in doubt. vLLM’s CUDA path captures a graph per batch size, so what these arms vary is the request dimension, which the TPU results explain with a data-structure argument (§4.1) and where no proposed optimization was going to pay. The token dimension, where this paper locates the only surviving effect, is not reached: no arm here varies tokens per step at fixed batch. The cross-architecture statement this table supports is therefore that **request-dimension padding is close to free on both architectures**, not that the premise is false on both — whether a GPU stack pays for token padding as a TPU stack does is untested, and is the experiment that would make the cross-architecture claim general. The comparison is also a bound rather than an equality: §4.1’s TPU statistic carries intervals of roughly ± 50 percentage points, so this table cannot resolve a difference between the architectures; what both support is that the paid share is small on each.

Table 7 is one measurement per arm. Repeating each arm 20 times on a fresh L4 and bootstrapping the position-of-n=9 statistic — $(c_9 - c_8) / (c_{16} - c_8)$, 10,000 resamples, median-based, the same convention as every other interval in this paper — resolves what the single measurement could not: with graphs, position is **16.0%** [15.0%, 17.1%]; under `-enforce-eager`, **0.7%** [-9.6%, 11.1%].

That 16.0% is not 16% of nominal padding paid. The denominator spans eight added *real* requests, of which n=9 carries one, so free-with-linear-real-work sits at **12.5%**, not 0%: graphs’ residual above the free reference is 3.5 points, agreeing to within half a point with the 4.0% obtained by differencing the arms. §4.2’s 23.1% has no real-work term in its denominator at all, so the two are not on a common scale and the apparent similarity is an artifact of reading both as percentages; Appendix §10.13 works this through. The single measurements, 16.7% and 12.8%, point the same way.

What is paid is the capture, and it is paid at startup. Enabling graphs costs 108 s of initialization, 118.7 s against 10.7 s, for a capture set fixed in advance. That is precisely the quantity BucketServe and LAPS manage when they write that the number of graphs must be limited, and it is a warmup cost. The TPU analogue is XLA compilation: 5–30 minutes for the first bucket and 30–120 s per additional one.

The launch sets no compilation-config override, so vLLM’s V1 default applies — piecewise capture, attention excluded since paged-KV-cache indexing is graph-unsafe — which, unconfirmed by a surviving log but if unmodified, would make this a shared mechanism with §4.1’s ragged kernel rather than a coincidental analogy: neither captured path sees the padded slots.

5 Optimizations designed against these measurements

Five interventions were designed against the compiled-shape premise. Each rejection below identifies a dimension the measurements show carries no cost:

- **Ladder placed against the workload** — works: -12.1% at n=2 and -46% p50 below the knee, at full memory; ahead of BucketServe’s objective at equal shape count by a margin §4.4 reports but does not establish the provenance of; -1.7% once prefix caching is on (§4.3–§4.4; prefix caching in Appendix §10.3).
- **Bucket-aware admission control** — premise false (§4.2).
- **Ladder redesign on the request dimension** — D1 does not exist; D3 inert by default.

Table 7: GPU control on an L4: CUDA-graph capture against eager execution. The increments are the measurement; the levels differ by a constant launch overhead.

arm	n=8	n=9	n=16	8→9	8→16	startup
CUDA graphs on	10.605	10.887	12.298	0.283	1.693	118.7 s
-enforce-eager	19.934	20.150	21.618	0.215	1.684	10.7 s

- **Last-chunk decomposition** — 20.6% worse measured (51.06 vs 42.33 ms).
- **Bucket-aligned step packing** — implemented twice: inert, then output-corrupting.

The one that works targets the single dimension the measurements leave open, and the rejections are informative rather than incidental. Three of them address the request dimension or per-request length padding, where §4.1 and §4.2 find no cost to recover, so their failure is what those sections predict; the fourth restructures work the stack already packs. The token dimension is the only place where the premise was not refuted by measurement, and the only place where an intervention produced a gain — though not for the reason expected, since the payoff was predicted to be confined to low concurrency and was not (Appendix §10.1).

A second positive measurement — release timing saving 26% of TPU time against stock at 25 req/s ($p=0.001$, six paired seeds) — is dynamic batching rather than a shape effect, reported as a re-measurement rather than a contribution. It is also a low-load effect that reverses to an 11.7% penalty by 70 req/s, and is not free once harness overhead is accounted for; Appendix §10.5 has the full arrival-rate sweep, the cost accounting, and the bucket-aligned-packing correctness failure the last bullet above draws on.

6 Validation, and what it does not cover

Every reported quantity is produced under a configuration contract that aborts a run when a controlled variable is left undeclared, is tied to a run identifier, and is recomputed from captured data by a script that exits non-zero if it disagrees with the text. §2 states the registered-prediction requirement that accompanies it.

No equivalent validation applies to explanations.

A proposed mechanism can be written, accepted, cited by later sections, and carried across successive drafts without any stage of the process being able to contradict it. Two instances in this paper show the consequence. The bandwidth account of free request padding persisted through four sessions of work before §4.1’s utilization measurement refuted it, and it was never supported by evidence so much as never confronted with any. The control cell of Appendix §10.1 was chosen on the reasoning that a 300-token prompt pads identically

on both ladders, which is true only while each request prefills alone; one scrape of the executed step distribution settled it, after both arms had run.

The consequence for a reader is that the measurements in §4 and the mechanisms offered for them do not carry the same weight, and we present them accordingly — with two qualifications. First, survival is not uniform: §4.2’s paid-share table survived the discovery that its rows were measured over different boundary sets as an *ordinal* claim, not a cardinal one, and Appendix §10.1’s rows above $n=2$ survive as floors rather than estimates under burst arrival, with only the isolated-dispatch control recovering interval-level statements — a value that reproduces while its interpretation is withdrawn has not survived in the sense the word usually carries. Second, the asymmetry is partly a property of what can be checked rather than of what is true: a measurement has a value a script can recompute, a mechanism does not, so it is never submitted to the check at all, and passing no check is not the same as passing one. Our own record limits the stronger reading — four of the errors we caught were instrument-definition failures, in which an analysis measured something other than its target, and those corrupt numbers rather than explanations, a class no guardrail here covers, so we cannot bound residual error in the measurements either.

7 Limitations

One TPU slice, one GPU, one primary model. A v5litepod-4 with a 4B model and a single L4. Appendix §10.4’s $TP=1/2/4$ ablation finds request padding cheap at every sharding and argues the regime map is set by batch size and dtype rather than parameter count, for dense weight-stationary decode only; model scale and multi-host topology stay unmeasured, and both change the fixed costs padding is absorbed into. A v6e-4 slice replicates the token-padding direction at three of four boundaries (Appendix §10.12) — a second data point, not a second interval. **The strongest single number is the least well supported:** the ~85% paid share at $n \leq 2$ rests on one boundary with no interval (§4.2).

Every ladder interval is within-boot. Table 9 measures a 4–11 ms between-instance floor, wider than any interval in §4.3 or §4.4, and each arm there is one boot. The placement result’s direction is robust — it survives placebo correction and reproduces across arm orders — but its precision is reported tighter than the design supports, and three boots per arm would be needed to state

it honestly as an interval.

Prefix caching is measured but not swept, and there is no production trace. Appendix §10.3 tests one prefix length, 2048 of 3000 tokens, on one workload shape: enough to show the placement target moves, not where it lands for a given amount of reuse. Everything outside §10.3 runs with caching disabled and so describes workloads with little prefix sharing — and caching is vLLM’s default. §4.2’s four length distributions are parametric families matched on mean length, not real traffic.

The ladder benefit has no measured upper bound in concurrency, and the recommendation is mediated by version-pinned internal flags. Appendix §10.1 sweeps 1 to 16 without a crossing, and above $n=4$ has no valid placebo, so those rows are floors rather than unbiased estimates. `VLLM_TPU_BUCKET_PADDING_GAP` and `ATTN_BUCKETIZED_NUM_REQS` are internal to vLLM and `tpu-inference` 0.25.0; the measurements survive a refactor, since they characterize a mechanism, but the operational advice does not.

Mixture-of-experts cost per padded token is unmeasured, no slice having been available — `v5litepod-4` was refused in thirteen zones across three continents, on-demand and spot, over a full day. The implementation settles the sign, not the size (Appendix §10.6); Appendix §10.6 also bounds the magnitude from public routing facts rather than hardware. `gpt-oss-20b` stayed unmeasured because of MXFP4 block alignment — weights pack 4-bit values in blocks of 32 and $2880/4 = 720$ is not a multiple of 32 at $TP=4$, while $TP=2$ cannot run on a four-chip mesh. `llama4` is out of reach at 109B.

The GPU control is inference-tier, one point rather than a curve, and co-located only. The L4’s arithmetic-intensity ridge sits elsewhere than `v5e`’s ~ 240 FLOP/byte, and since the ridge sets the batch size at which token padding stops mattering (Appendix §10.4), these thresholds should not be transferred without re-measurement. Startup was measured at vLLM’s default capture set and the number of captured shapes was never varied — precisely the axis `BucketServe` and `LAPS` trade along. On a disaggregated deployment the padding question divides in two.

Prefill step cost above $n=16$ is not isolable, and at $n=16$ the clean sample is 7–11 dispatches per arm. **The $n=4$ convergence is unexplained and is not an operator effect:** device time moves smoothly through $n=4$, and the observations that appear to converge there are not independent — `LENS`’s failure and the paid-share drop are the same quantity twice.

8 Related work

Pope et al. derive the memory-bound to compute-bound transition for transformer inference on TPU analytically; §4.1’s byte accounting is a measured instance of that regime rather than a discovery. We do not claim that the weight-load floor explains free request padding, since §4.1 tests that account and rejects it.

RPA is the technique this work validates: per-request padding costing nothing is what its ragged-tiling design predicts, though it does not discuss the request-count dimension, report cost against batch size, or quantify how much padding survives it. **LENS** (Appendix §10.4) supplies the model form; we supply the hardware it was not tested on and the ablation showing that its length term is not what carries its reported accuracy.

PagedAttention/vLLM is the stack measured throughout; **Orca**’s iteration-level scheduling produces the per-step batches; **Sarathi-Serve** introduced the chunked prefill that §4.2 controls for. **DistServe** and **Splitwise** disaggregate prefill from decode, which is the architectural response to the finding that variance is a prefill phenomenon, and which bounds our advice to co-located deployments. **Vidur** established simulator-fidelity validation as the standard for simulation-based serving studies; our holdout discipline follows it.

SGLang and **TensorRT-LLM** implement their own shape and graph handling, with different capture policies and, in `TensorRT-LLM`’s case, an ahead-of-time engine build with explicit optimization profiles. Nothing here is measured on either, and the claims should be read as applying to vLLM-style designs, in which shapes are compiled or captured from a ladder the serving loop rounds up to.

Shape-polymorphic and bounded-dynamic-shape compilation — `XLA`’s dynamic dimensions, `torch.compile`’s dynamic shapes, `TensorRT`’s optimization profiles — are the alternative to a fixed ladder, removing the boot-time cost a ladder pays for coverage. Our finding argues against investing there for this workload: with run-time padding close to free on two of three dimensions and the ladder’s own cost already boot-time and already shrinking under a persistent cache, dynamic shapes trade a cost that already amortizes for flexibility this workload does not need.

BucketServe and **LAPS** manage length-bucketing overhead on GPU; `LAPS` is compared against by argument in this section, not run, since no artifact was available. §4.4 runs `BucketServe`’s own ladder-design objective on this stack rather than arguing against it — an objective-level comparison, not a reimplementa-tion of their system: solved globally and given the same shape budget, its ladder is 1.61 ms slower than one chosen to minimize absolute padded tokens, and both are

about 17–19 ms faster than the stock ladder. Their ladder design is therefore effective, and our disagreement with these systems is narrower than a premise-level objection — it concerns which dimension the padding occupies (the request dimension is free, §4.1, and per-request length padding does not exist, §4.2), and, for the token dimension where it is real, which objective converts to time. The cross-architecture comparison itself is measured rather than asserted, and is reported as a result in §4.5 rather than as related work.

How, then, do prior bucketing techniques achieve their reported speedups? If the padding premise is false on both architectures, systems reporting end-to-end improvements from bucketing are improving something else — and §5’s release-timing re-measurement supplies a candidate: dynamic batching, an arrival-and-composition effect rather than a shape effect, worth something independent of padding even though its magnitude does not survive scrutiny (Appendix §10.5). The defensible claim is the negative one — **a bucketing result that does not control for batch composition cannot distinguish a shape effect from a scheduling one** — and it rests on the premise measurements of §4.1 and §4.2.

9 Conclusion

The two accelerator families tested here reach the same design by different routes: XLA compiles a ladder of shapes, and CUDA captures a graph per batch size. Both round every step up to the nearest entry, and the optimization literature treats that rounding as a cost to be recovered. It is not, on either of the two chips measured — a batch just above an entry costs what the entry below costs, because a ragged attention kernel does almost no work for slots holding no key–value blocks (under 0.7 μ s per slot, against 27.5 μ s if it were paid), and a captured graph does not care that part of its batch is unused.

What shape coverage costs is a boot-time budget, and only part of it amortizes: a persistent cache erases most of the 108-second CUDA-graph capture and 5–30 minute XLA compilation on every restart, but not the memory headroom a finer ladder demands before the server starts at all (§4.3) — the quantity BucketServe and LAPS manage by limiting graph count. Reducing shape count is worth doing for time-to-serve; routing requests to avoid run-time padding is not.

The exception is the token dimension: real arithmetic, paid at 23.1% of nominal at batch 4, indistinguishable from zero by 16, and substantial at low batch — 55–75% at $n=1$ across three boundaries with intervals on a second TPU generation (Appendix §10.12), better-supported evidence than the single-boundary, interval-free $\sim 85\%$ point estimate at $n \leq 2$ on the first — and that

low-batch regime (interactive serving, tight-latency deployments, the prefill half of any disaggregated system) is where the ladder buys something. **The variable that pays is placement, not shape count.** A fourteen-shape ladder adding one entry the default lacks reduces end-to-end latency by 12.1% at the prompt length that straddles it, gains nothing at a length it does not, and boots at the stock memory fraction with unchanged cache capacity; a twenty-one-shape ladder achieves the same reduction but will not start above a memory fraction of 0.85, costing 8.8% of cache capacity and 53% more startup — every cost measured scales with cardinality. The ladder can be chosen offline: expected padding computed from a length distribution predicted the measured reduction to within 5%, so the design rule is the finest ladder that still boots, with entries placed against the distribution the server actually prefills — a latency gain below saturation rather than added capacity, shrinking as prefix caching moves the prefill onto a different compiled entry (Appendix), so entries must target uncached prefill lengths.

Three predictions registered before measurement failed, and each failure was worth more than a confirmation would have been — the paper’s methodological result: every reported measurement has survived revision, while four of the last five retractions were claims about mechanism, because a measurement passes through machinery that can reject it and an explanation does not. The remedy is to require a falsifiable number before hardware is provisioned. It is cheap, and it is the practice we would most recommend carrying into other measurement work.

10 Appendix

Supporting material not required to evaluate the paper’s four core results (§3, §4.1–§4.3, and their offline design rule in §4.4): the registered-prediction concurrency sweep, the deployment-value bounds, the prefix-caching interaction, the supporting ablations §4.1’s mechanism draws on, the release-timing measurement in full, and the mixture-of-experts source-reading account. Section numbers below restart at 10.1 for this reason, not because the content is new.

10.1 The benefit does not decay with concurrency, contrary to a registered prediction

Because padding is paid at small batch and free at large (§4.2), the benefit of §4.3 appears as though it should be conditional on load. That reasoning predicts a crossing, and it was registered before measurement: the difference should shrink monotonically and reach zero between $n=4$ and $n=16$. Sweeping concurrency from 1 to 16 on both ladders, in both arm orders, does not sup-

port it. Placebo-corrected differences in milliseconds, negative where the finer ladder is faster:

There is no crossing. End-to-end latency is 3.5% to 12% lower at every concurrency sampled, the effect is non-monotone rather than decaying, and at $n=16$ it is larger in absolute terms than at $n=1$.

Why the prediction failed The reason the prediction failed is visible in what the stack executes. The prediction reasoned about an individual request’s padding shrinking as more requests share a step. Under chunked prefill the scheduler admits requests in waves and packs them, and the size of the packed step, rather than any request’s length, selects the compiled shape. Snapshotting the `iteration_tokens_total` histogram around one $n=16$ cell shows a single repeat costing about three prefill steps, landing in $(256,512]$, $(512,1024]$ and $(2048,4096]$, alongside 93 decode steps of at most 16 tokens. The two arms produce near-identical step distributions, differing only in which compiled shape those steps round up to. Padding is therefore transferred from individual requests to the packed step rather than eliminated.

Tension with the paid-share curve This is in tension with §4.2, which finds the paid share falling to indistinguishable from zero by batch 16. The two measurements differ in what is held fixed: §4.2 varies batch size at a fixed boundary and attributes padding per request, while this sweep varies offered concurrency and allows the scheduler to choose step composition. If both are correct, the reconciliation is that per-request padding vanishes while per-step padding does not, and that a ladder acts on the second. Testing this on this data requires padded tokens per packed step, which the available instrument cannot supply: `iteration_tokens_total` bins on powers of two, which is coarser than the ladder spacing under comparison. A step recorded in $(2048,4096]$ pads to 2048, 2560, 3072, 3584 or 4096 depending on where in that bin it falls, and the two ladders differ precisely inside the bin. A step-scoped instrument built later and run on a second TPU generation supports the reconciliation directly rather than resolving this tension on this data: Appendix §10.12’s per-step overhead stays in a 33–59% band from $n=1$ to $n=15$ rather than falling toward zero the way the per-request measure does.

A defect in the control cell One design defect bounds these results. Prompt 300 was intended as a placebo at every concurrency, on the reasoning that it pads to 512 on both ladders. The step histogram shows this holds only while each request prefills in its own dispatch: at $n \geq 4$ the packed step lands where the ladders differ, so the cell is treated rather than inert. Its measured difference is correspondingly unstable across arm orders at $n=8$ and $n=16$ (–6.0 against –27.7 ms, and –1.5 against –17.2 ms)

where the treated cells are stable. The $n \geq 4$ figures above are therefore floors on the effect rather than unbiased estimates, and arm order is their only control.

An isolated-dispatch control The defect above is that concurrency does two things at once: it makes the regime realistic, and it lets the scheduler co-schedule prefills. Only the second breaks the control. Releasing requests on a 120 ms stagger separates them: a 3000-token prefill takes roughly 90 ms here, so each prefill runs alone in its step, while a 32-token decode takes about 160 ms, so requests still overlap in decode and the concurrency is real.

Isolation was verified rather than assumed. Scraping the step-size histogram around eight staggered requests at prompt 300 shows **eight separate prefill steps, all in the (256, 512] bin**; the same eight released as a burst produce three steps, in $(256,512]$, $(512,1024]$ and $(1024,2048]$. The stagger therefore restores the per-request ladder mapping the control depends on.

A registered prediction that the control cell would return to zero was wrong: it shows a stable offset of 4 to 11 ms. Because 300 tokens pads to 512 on both ladders once prefills are isolated, and the histogram confirms they are, no ladder effect can reach that cell, so the offset is a difference between server instances and is exactly the quantity a control exists to measure. Subtracting it, prompt 1200 sits at zero as the ladders predict, since both pad it to 2048, and prompt 3000 carries a large reduction that does not decay through $n=16$. Intervals here are ± 0.7 ms rather than floors.

The effect under isolation is substantially larger than under burst arrival, at 66 to 108 ms against the 18 to 50 ms of the table above. That is the same mechanism seen from the other side: a packed step amortizes one padding charge across several requests, while an isolated prefill pays its own in full. **The claim that the benefit does not decay through $n=16$ therefore holds at interval level under isolated dispatch, and as a floor under burst arrival.**

The sign survives this; the magnitude does not. The placebo’s spread across arm orders at $n=8$ and $n=16$ is approximately 20 ms, comparable to several treated differences in the same table, so no $n \geq 4$ magnitude is resolvable at interval level. What is resolvable is direction: the raw differences at $n=16$ are –32.2 and –46.3 ms, and subtracting even the most extreme placebo estimate observed leaves both negative in both arm orders. The absence of a crossing is established at $n=1$ and $n=2$, where the placebo is valid; above that it is an observation with a floor.

Table 8: Placebo-corrected latency difference against concurrency, in milliseconds. Negative values favour the finer ladder.

prompt	n=1	n=2	n=4	n=8	n=16
1200	-9.9	-18.0	-8.9	-16.2	-23.0
3000	-18.7	-37.3	-21.5	-50.4	-37.0

Table 9: Ladder difference with prefills isolated by a 120 ms arrival stagger. Prompt 300 pads to 512 on both ladders and measures the offset between server instances; 1200 pads to 2048 on both and should show no effect.

n	prompt 300 (offset)	prompt 1200, corrected	prompt 3000, corrected
4	-11.2 ms	+8.2 ms	-65.8 ms
8	-9.7 ms	+1.2 ms	-108.1 ms
16	-4.5 ms	+2.7 ms	-98.1 ms

10.2 The gain is a latency reduction below saturation, not additional capacity

The results above are latencies at fixed, small concurrency, while the cost of a longer ladder is denominated in cache tokens. A deployment choosing whether to spend memory on shapes requires the conversion. Table 10 sweeps offered load open-loop against both ladders, at the stock 0.92 fraction where both boot, with 60 requests per rate at prompt 3000:

Both ladders knee between 8 and 12 req/s and saturate near 14. Sustained goodput rises from 14.03 to 14.40 req/s, or 2.6%, so the padding removed was not entirely slack. But 2.6% is far short of what "padded tokens are real arithmetic" implies for a 25% reduction in the prefill shape, and it is not where the effect is concentrated. The effect is concentrated just below the knee: at 8 req/s the placement ladder is 46% faster at p50 and 39% faster at p95.

That shape is consistent with §4.2 and repairs part of the tension in Appendix §10.1. A saturated server packs prefills to the `max_num_batched_tokens` budget, so padding is amortized across a full step and the ladder has little influence, which is what the paid-share curve predicts. Below saturation the steps are smaller and closer to per-request, and the ladder entry accounts for most of the step’s cost. Appendix §10.1’s persistent benefit was measured under burst arrival at fixed concurrency, which loads the server differently from a steady arrival process at the same nominal rate.

For the fourteen-shape ladder this trade is favorable, since it costs no capacity at all: four additional compiled shapes reduce tail latency by up to 46% in the regime most interactive deployments run in. For the twenty-one-shape ladder, which costs 8.8% of capacity, the same arithmetic argues against it, since surrendering 8.8% of the cache to gain 2.6% in sustained throughput is an unfavorable exchange at saturation.

10.3 Prefix caching moves the target the ladder is placed against

Prefix caching is disabled elsewhere in this work, and production vLLM enables it by default. It removes already-computed prefix tokens from the prefill, so the step lands on a different compiled shape than the request’s length implies, which bears directly on a recommendation about where to place entries.

Testing this requires a workload with a cacheable prefix. Requests built from independent token sequences share nothing, so a cache cannot hit them, and an experiment run against such a workload would report caching having no effect while saying nothing about production. Here every request shares a fixed 2048-token prefix and varies only its 952-token tail, which is the structure of a system prompt or a few-shot preamble. Prompt 3000, two concurrent requests (Table 11):

Effect of prefix caching on the placement benefit The placement benefit falls from 35.9 ms, or 12.3%, to 3.0 ms, or 1.7%. A prediction registered before measurement anticipated this, and for the stated reason: with 2048 tokens cached the server prefills approximately 952, which pads to 1024 on both ladders, while the two ladders differ only at 3072 against 4096. The entry chosen in §4.3 is no longer the entry the step uses. The cached-token counter is the arm’s own evidence that the treatment applied, since 43,008 is exactly 21×2048 , so twenty-one of twenty-two requests hit the prefix and the first populated it.

The mechanism is unchanged and the operating point has moved. Caching does not make padded tokens cheap; it removes tokens from the prefill, and a ladder placed against prompt length is then placed against the wrong distribution. **Entries should therefore be placed against the distribution of uncached prefill lengths.** This also bounds §4.3 and Appendix §10.2, which are measured with caching disabled and so describe workloads with little prefix reuse. On this workload caching is worth considerably more than the ladder: 292.5 ms to 181.5 ms, a 38% reduction, against the 12.3% the best placement achieves without it.

Table 10: Sustained goodput and latency against offered load, both ladders at the stock memory fraction.

offered req/s	goodput (default → gap1024)	p50 ms	p95 ms
2	2.30 → 2.30	286 → 246	504 → 318
4	4.59 → 4.59	326 → 256	758 → 496
8	9.03 → 9.08	974 → 529	1622 → 996
12	12.45 → 12.68	2088 → 1895	3471 → 3093
16	13.45 → 13.84	2571 → 2275	3570 → 3418
24	14.03 → 14.40	3026 → 2921	3881 → 3727

Table 11: Ladder placement with and without prefix caching, over a workload sharing a 2048-token prefix.

ladder	caching	e2e	prompt tokens cached
default (10)	off	292.5 ms	0
gap 1024 (14)	off	256.6 ms	0
default (10)	on	181.5 ms	+43,008
gap 1024 (14)	on	178.5 ms	+43,008

Table 12: LENS against a constant-only model, held-out mean absolute percentage error.

batch size	LENS	constant-only
1	0.38%	0.96%
2	0.39%	0.86%
4	19.77%	14.80%

10.4 Supporting ablations and controls

A published latency predictor A published latency predictor, reproduced and scoped. LENS predicts NPU inference latency to 2.15% mean absolute percentage error (MAPE) using a per-bucket `intercept + slope × length` fitted from two end-to-end measurements per bucket. Reproducing its protocol on TPU across 5 buckets × 3 batch sizes, with 7 repeats per point and a mid-bucket point withheld from each fit, gives MAPE 5.23% and a worst cell of 22.4%, near-perfect at n=1–2 and failing at n=4. Replacing the model with a constant, the mean of the two calibration points:

At n=1–2 the within-bucket curve is nearly flat, at a flatness of 0.97, so any two-point fit is near-perfect. LENS does beat a constant there, at 0.38% against 0.96%, but the gap is 0.6 percentage points on errors already below 1%, and it is not evidence that the model form transfers. At n=4 the length term is actively harmful. The failure is not an artifact of which points were fitted: over all three choices per cell the n=4 error varies by up to 44.8 percentage points, but its minimum is still 16.97%, far above LENS’s reported 2.15%. Batch sizes above 4 were not measured, because this experiment reproduces LENS’s own protocol, which is specified over bucket structure rather than batch size.

Decode cost against batch size Decode cost against batch size. With `prompt_len=256` and `output_len=64`, measured across the full compiled request ladder:

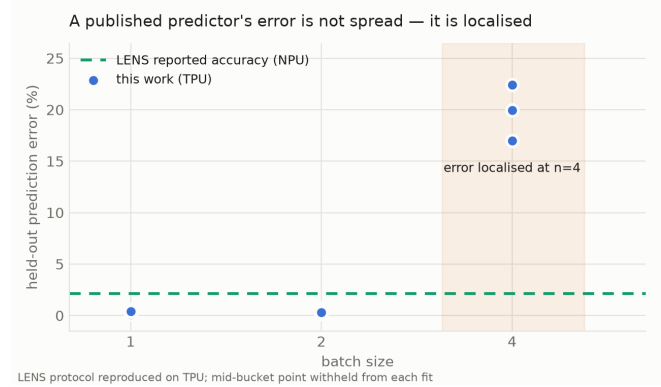


Figure 2: Held-out prediction error against batch size, with LENS’s reported 2.15% as a reference line and the failure region shaded. The claim is not that the predictor is inaccurate; it is that the error grows sharply by n=4. Batch sizes 1, 2 and 4 are sampled, with the failure at the endpoint, so these data cannot say whether accuracy returns above 4.

There are two regimes and the boundary between them is sharp. Below $n \approx 8$ the step barely moves, with batch rising eightfold for 1.22 times the cost. Above $n \approx 32$ the step is nearly linear in batch, at 1.67, 1.82 and 1.86 times for successive doublings, and per-sequence cost flattens at roughly 200 μ s. Queue time at n=128 and n=256 means those columns are not clean wide batches and no claim depends on them.

Distribution of device time Where device time is spent. An operator profile is a direct observation, which the roofline is not. Share of TPU device time:

Matmuls dominate at low batch and give way to attention as key-value state grows, while collectives hold a flat 13.4% whose latency component no roofline models. Nothing moves discontinuously at n=4.

An invalid microbenchmark One microbenchmark does not measure the quantity it appears to. An isolated matmul at the model’s real sharded shapes returns 142.9 μ s at M=1 and 143.6 μ s at M=256, a flatness that can be read as the weight-load floor with confounds removed. The qkv projection holds 7.86 MB per chip, so the bandwidth floor is 9.6 μ s: the measurement sits fifteen times above it, at 7% of peak, and is timing per-dispatch overhead. Amortizing over a loop reports 1250% of peak be-

Table 13: Decode cost against batch size across the compiled request ladder.

n	1	8	32	64	128	256
ms/step	4.02	4.91	9.19	15.32	27.90	51.83
$\mu\text{s}/\text{step}/\text{sequence}$	4020	614	287	239	218	203
queue (ms)	0.0	0.0	0.0	0.1	62.2	298.3

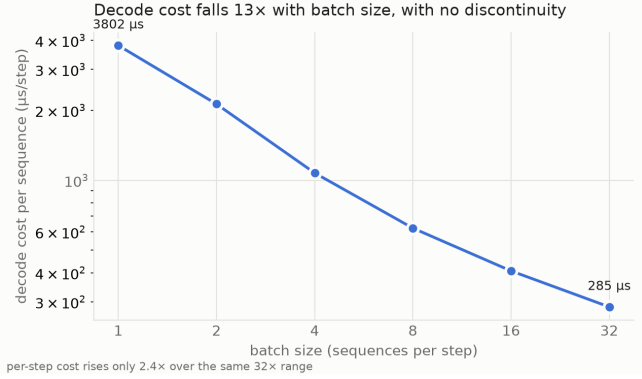


Figure 3: Decode cost per sequence against batch size, log-log.

Table 14: Share of TPU device time by operator category.

n	attention	collective	matmul/fusion
1	6.8%	13.5%	78.5%
4	15.4%	13.9%	69.6%
16	34.2%	13.4%	51.4%

cause XLA hoists the loop-invariant matmul; chaining iterations is physically valid at 79% of peak but streams weights from high-bandwidth memory (HBM) every iteration and remains bandwidth-bound.

Per-dispatch variance Per-dispatch variance is a prefill phenomenon. Over 9 repeats on the same server, decode spread is $1.00\text{--}1.04\times$ at most batch sizes, while prefill is $1.00\text{--}1.03\times$ at $n \leq 4$ and $1.18\text{--}1.26\times$ at $n \geq 8$. Variance appears exactly where the scheduler begins splitting dispatches, and decode, which makes no chunking decision, never exhibits it. This localizes the effect without explaining it. The aggregate figure does not describe every cell: bootstrapping the decode cells §4.1 depends on gives 95% interval widths of 38.7% at $n=8$ and 28.2% at $n=9$ over 21 repeats, wider than several differences a reader might otherwise treat as signal.

Sharding ablation Free request padding is not an artifact of the sharding. Model, chips and workload were held fixed while tensor-parallel degree alone was varied. The prediction was registered before measurement: per-chip weight bytes scale as $1/\text{TP}$, so the level should scale with $1/\text{TP}$ and the shape should be preserved.

Both halves of the prediction were wrong. The level

scales sub-proportionally, which the roofline cannot explain because it does not model the inter-chip collectives the higher-TP arms pay, and the curve becomes flatter with less sharding rather than preserving its shape. Both errors follow from a single omitted term. Fitting $T(\text{TP}) = W/\text{TP} + F$, a weight load that shards plus a fixed cost that does not, gives $T(\text{TP}) = 2.48/\text{TP} + 0.38$ normalized to the $\text{TP}=4$ step, with fitted values 1.00, 1.62 and 2.86 against measured 1.00, 1.63 and 2.86. Fixed, non-sharding cost is 38% of the $\text{TP}=4$ step, roughly 1.5 ms at $n=1$, and that single term explains both the sub-proportional level and the flattening. The fit has two free parameters against three points, one of which is fixed by normalization, so it carries a single genuine degree of freedom and the residual overstates how much evidence it provides.

F is not the collectives: a constant term cannot represent inter-chip communication, which is zero at $\text{TP}=1$ and largest at $\text{TP}=4$ and therefore scales with TP , whereas F by construction does not. The ablation nonetheless answers its question: if request-dimension padding were cheap only because that dimension is not the bottleneck at $\text{TP}=4$, reducing TP would expose it, and instead padding is cheapest at $\text{TP}=1$ where the fixed cost is largest.

Model scale, and dtype as the lever Model scale does not move the regime map. A second registered prediction failed here. TinyLlama-1.1B has a 3.6-fold smaller per-chip weight floor than Qwen3-4B, so if the mechanism were that floor its paid share should be higher. It is lower, at -1.2% and 13.4% at $n=4$ and -1.1% and 5.9% at $n=8$. Model size is the wrong lever: for dense weight-stationary decode, bytes $\approx 2 \cdot \text{params}$ and FLOPs per token $\approx 2 \cdot \text{params}$, so both models sit at 1.00 FLOP/byte/token and arithmetic intensity is the batch size, independent of parameter count. The ridge is a property of the chip ($v5e: 197 \text{ TFLOP/s} \div 819 \text{ GB/s} \approx 240 \text{ FLOP/byte}$), and shrinking the model shrinks the floor and the work in the same proportion. This answers the single-model limitation analytically for that regime, and breaks for mixture-of-experts models, where bytes scale with distinct experts touched, and at long context, where key-value state rather than weights sets the floor.

Two cautions apply. The identity is derived for decode, while the paid-share numbers are prefill; applying a decode argument to prefill measurements crosses between two of the quantized dimensions, which §3 states this paper does not do, so the numbers are reported as mea-

Table 15: Tensor-parallel ablation. Both halves of the registered prediction fail, and a single omitted fixed term explains both.

TP	per-step level vs TP=4	predicted	cost rise, n=1→32
4	1.00×	1.00×	2.33×
2	1.63×	2.00×	2.41×
1	2.86×	4.00×	1.83×

Table 16: Release-timing policies against arrival rate, positive being a saving in TPU time against stock.

req/s	wait	hybrid	hybrid p95
10	+36.2%	+29.0%	-32.6%
25	+22.2%	+22.8%	+28.9%
40	+7.1%	+17.6%	+21.5%
55	+7.2%	+11.6%	-6.6%
70	+2.3%	-11.7%	+18.7%

surements with the explanation withdrawn. The intensity argument also explains why model size should not move the paid share, not why it moved downward; the likely reason is that non-weight fixed costs form a larger fraction of a 0.55 GB model’s step.

Dtype is the lever that should move the token dimension. W8 weights halve bytes and leave FLOPs unchanged, doubling intensity per token and moving the crossing from batch ≈ 240 to ≈ 120 . That yields a registered prediction — under W8 the token-dimension paid share at a fixed boundary rises — which **has not been run**. It is future work rather than a result, and nothing above depends on it.

10.5 Release timing, in full: the low-load effect, its cost, and the packing bug

It is a low-load effect that reverses. Swept across arrival rate against stock, positive being a saving:

The saving decays monotonically with load and becomes an 11.7% penalty by 70 req/s. A single measurement at 25 req/s cannot distinguish that from a robust effect, and it samples the most favorable region of the curve.

It is also not free. The harness scrapes `/metrics` around every batch, adding a median 22.6–24.9 ms of inter-dispatch overhead to a policy that never waits. That inflates stock’s p95 from roughly 24 ms to 86 ms and conceals the delay the waiting policy introduces deliberately. Measured under that harness the cost appears as +14.8% p95 ($p=0.570$); simulated on an efficiently driven server the same policy costs approximately +188% p95 at 25 req/s. The defensible statement is narrower: the hybrid policy reaches nearly all of wait-to-fill’s saving, at 30.2% against 31.9%, for a small fraction of its latency cost, at +188% against +1461%. The result is a point on a cost–latency trade-off curve.

The cost model does not survive a wider sweep.

Holding out rates it was never fitted on gives 4.9% MAPE overall but a worst cell of 19.7% (hybrid at 55 req/s), against this project’s own 15%-per-cell rule, so it fails. The earlier holdout varied neither rate nor prompt length and so could not detect an error constant across them. The simulated policy numbers are internally consistent predictions that do not transfer to unseen load.

Bucket-aligned packing requires a separate note. The second implementation measured -29% TPU time and -49% p99; the correctness gate then showed that 4 of 48 greedy completions differed, every one at a prompt length just above a bucket boundary. The patch was silently dropping prompt tokens. Trimming `num_scheduled_tokens` after the scheduling loop leaves the request’s bookkeeping untouched, so deferred tokens are skipped rather than rescheduled, and the step is cheaper because it does less work.

10.6 Mixture-of-experts padding, from source

What the implementation settles is the direction. In `tpu-inference 0.25.0` a padded token entering the fused expert kernel carries whatever the routing top- k returned for its unwritten row, and is dispatched to that many real experts — eight, for Qwen3-30B-A3B. Two costs follow, and only one has a dense analogue. The first is one token’s worth of expert compute, which is what a real token pays too, and which therefore leaves the paid *share* unchanged, that share being a ratio. The second is that a padded token can activate an expert no real token in the step required, adding a group to the grouped matmul that would not otherwise run. That second term grows as the batch shrinks, which is exactly where padded share is already highest (§4.2), and it is the quantity Appendix §10.4’s arithmetic-intensity argument already names as the reason that argument does not extend to these models: bytes there scale with distinct experts touched, and padding is a way of touching more of them. The backend carries a flag, `MOE_ROUTE_PADDING_TO_EXPERT0`, that collapses padding onto a single expert at zero weight; it ships disabled, is inert when attention is data-parallel, and fails open, serving the unmitigated path if it cannot read the step’s valid-token count.

The sign survives this; the magnitude does not. Whether the second term is three percent of a padded token’s cost or sixty decides whether that flag is an operational

recommendation or a footnote, and no reading of the source answers it.

Verified against the pinned wheel directly (`tpu_inference==0.25.0`, not inferred): `jax.lax.top_k` runs on every row's real router logits before the flag does anything — `topk_indices = jnp.where(token_valid, topk_indices, 0)`, weight forced to 0, happens strictly after selection, gated on the same `query_start_loc`-derived boundary §10.6's main text already describes. The flag neutralizes a padded row's *dispatch*, not its routing computation, confirming the existing description rather than correcting it. What this does not settle: whether a padded row's *input* to that computation is a fixed, deterministic value (a pad-token embedding, reproducible run to run) or carries forward whatever a KV-cache slot held from a previous occupant, which would make routing for that row depend on request history rather than being a pure function of the current step's real inputs. That traces buffer-reuse semantics this source dive did not reach, and is a correctness question, not only a cost one, independent of the magnitude bound above.

A bound from public routing facts, not from source or hardware. Qwen3-30B-A3B routes among 128 experts, 8 per token, trained with the standard load-balancing auxiliary loss that GShard, Switch Transformer and ST-MoE all use, minimized under uniform expert utilization (Qwen3 Technical Report, arXiv:2505.09388). Treat per-token routing as a first-order i.i.d. uniform 8-of-128 draw — the target that loss trains toward, not a measurement of real traffic — and the expected fraction of a padded token's 8 dispatches that land on an expert none of n real tokens in the step already touched is exactly $((128-8)/128)^n$, by linearity of expectation (`scripts/o11_moe_padding_bound.py`, self-tested). This reproduces both of §10.6's own stated bounds at batch sizes §4.2 tests directly: **60% at $n=8$** , the paper's own upper anchor, and **3% at $n \approx 54$** , past every batch size this paper measures, with 36% still remaining at $n=16$. The bound is on how many *new matmul groups* a padded token causes, not on microseconds — converting that to time needs the fixed per-group launch/weight-load cost against the marginal cost inside an already-active group, which is a hardware question this estimate cannot answer — and it assumes i.i.d. uniform routing per step, which the auxiliary loss trains toward on average but does not guarantee for one step's real traffic: correlated routing (similar prompts routing similarly) would push the true fraction below this estimate, narrow-domain traffic could push it above. What it settles is that the "sixty" anchor is not a worst-case artifact — it is what padding's own regime (small batch, where §4.2 shows it is paid at all) implies under the routing behavior the architecture is trained to have.

Table 17: Utilization against batch size, testing the memory-bandwidth account of free request padding. The account predicts about 49% MFU at the $n=256$ the prediction names; the refutation is stated at $n=64$, the largest genuinely full batch.

n	1	8	32	64
MFU	0.3%	1.7%	3.6%	4.4%
high-bandwidth memory (HBM) utilization	61.4%	52.1%	31.2%	21.4%

10.7 The memory-bandwidth account, tested and rejected

The memory-bandwidth explanation is tested and rejected. A natural account of free request padding is that the step reads the whole weight set regardless of batch, so padded slots are absorbed into a fixed cost that batch size does not change. That account predicts utilization climbing toward the compute roof past the arithmetic-intensity ridge near 240 FLOP/byte, reaching approximately 49% at $n=256$. The quantity is **model FLOPs utilization (MFU)**: the arithmetic the model performs, divided by the arithmetic the chip could perform in the same time. A value of 5% means the chip spent 95% of its time doing something other than the model's floating-point work — waiting on memory, mostly. The table also reports utilization of high-bandwidth memory (HBM), the off-chip memory the weights are streamed from. Measured with `prompt_len=256` and `output_len=64`:

The $n=256$ cell is deliberately not tabulated. At that offered concurrency the server has not admitted all 256 requests into the batch — some are still queued — so the column would not describe a batch of 256, and a contaminated cell in a table outlives the sentence disclaiming it. For the record it read 5.1% MFU and 11.1% HBM utilization, both further from the predicted 49% than $n=64$ is, so excluding it costs the refutation nothing. The refutation is stated at $n=64$, where queue time is 0.1 ms and the batch is genuinely full.

MFU here is $2 \times \text{parameters} \times \text{tokens}$ against the chip's 197 TFLOP/s bf16 peak, with attention FLOPs excluded, which is the same dense weight-stationary accounting used for the intensity argument. Two caveats apply. MFU during memory-bound decode is close to a tautology, since it restates step time against a fixed numerator; and it is used here only to falsify a prediction expressed in the same units, not as a figure of merit.

A memory-bound step is one whose achieved bandwidth sits near the roof. This one falls monotonically to 21.4% by $n=64$, where the queue is 0.1 ms and the column is clean, so the account fails without recourse to the contaminated cells. Neither the compute-bound nor the memory-bound account is supported. One use of the roofline remains valid, namely byte accounting: the

step reads 2.01 GB of weights regardless of batch size. Achieved bandwidth, however, is computed as bytes divided by measured time, and therefore restates the step time from which it is derived.

Does low utilization mean most of the step is unaccounted-for overhead? Not by the evidence this paper has. Low MFU and HBM utilization are claims about efficiency relative to the chip’s peak, not claims that device time goes unexplained: §10.4’s operator profile (Table 14) accounts for essentially all device time as attention, collectives and matmul/fusion (98.8–99.0% of the total at every n it covers), leaving little room for a large, separate dispatch-overhead category at those batch sizes. That table stops at $n=16$, though, short of $n=64$ where the roofline account above is cleanest, so this bound is suggestive rather than a direct measurement of the same cell. Separately, the TP ablation’s fixed non-sharding term (F , 38% of the TP=4 step at $n=1$) is a different decomposition — by TP-scaling, not by operator category — and this paper does not establish whether F sits inside one of Table 14’s categories or represents time neither table’s method would see; doing so needs a per-operator profile at the same n and TP the ablation measures, which was not collected. The discriminating experiment in §4.1 defends the request dimension against this specifically regardless: a $32\times$ reduction in compiled slots moves the fixed term 2%, which bounds the per-slot cost directly rather than inferring it from a utilization ratio, so §4.1 does not depend on this budget being closed. The abstract’s and conclusion’s general framing does, to the extent a reader takes “free” as a property of the hardware rather than of this stack’s measured overhead profile — which is why §7 already scopes the claim to one slice, one GPU and one primary model, and why that scope should be read as load-bearing here specifically, not only as a general caveat.

10.8 Measurable range and the request launcher

The upper limit of what can be measured was partly a property of the measurement apparatus. A naive launcher makes this quantity appear unmeasurable above $n=8$, because the scheduler splits every dispatch. Splitting tracks request count rather than token count — a dispatch of 8192 tokens at `max_num_batched_tokens=8192` never splits, while one of approximately 1024 tokens at $n=8$ splits half the time — which is the signature of an arrival race rather than a capacity limit. Releasing all requests from a thread barrier after connection setup reduced arrival spread by a factor of 7.6 at $n=32$, from 15.4 ms to 1.7 ms:

The real barrier lies between 16 and 32 rather than at 8. Under a synchronized launch $n=16$ becomes measurable, and the paid share there is indistinguishable from zero across three boundaries that each double the padded to-

ken count.

10.9 Padded share under four length distributions

Padded share is governed by where a distribution’s mass falls relative to the compiled boundaries, and not by its dispersion. Four prompt-length distributions were served at a single Poisson arrival rate (8 req/s, `output_len=64`, 120 requests each), reporting time to first token (TTFT) and inter-token latency (ITL). Their parameters were solved so that every family has the same mean length of 1000 tokens, so equal request rate means equal offered tokens and distribution shape is the only quantity that differs:

The padded share spans 8.5% to 33.3%, a factor of four, at equal offered tokens.

Dispersion as a predictor of padding Coefficient of variation does not order it. The three dispersed families run CV 0.58, 0.91 and 1.51 against padded shares of 27.4%, 33.3% and 28.0%, which is not monotone. Raggedness is therefore not the variable that governs padding, which is the assumption the bucketing literature rests on. What governs it is where the distribution’s mass sits relative to the compiled boundaries: the fixed-length arm pads least because 1000 tokens sits just below the 1024 entry, so little is rounded away, and the same family would pad far more at 1100.

A retracted ordering A prediction registered before this measurement was wrong, and its failure retracts the explanation that motivated it. The same four families measured without matching offered tokens — mean lengths of 256, 384, 704 and 2056, so the uniform arm carried about eight times the tokens of the fixed arm — placed the fixed-length family highest, at 51.0% against 27.3% for uniform. The explanation offered for that ordering was that a fixed length just above a boundary pads every step by the same large amount while a spread distribution averages across buckets, and the prediction registered here was that the ordering would survive matching. It does not: at equal offered tokens the fixed family pads least of the four. The ordering followed from unequal token load, and the explanation built on it does not stand.

This is the conclusion of §4.3 reached from the other direction. There, ladder *placement* rather than shape count determines what a ladder buys; here, distribution *position* rather than dispersion determines what a workload pays. Both identify the same governing quantity: where lengths fall relative to boundaries.

Table 18: Fraction of dispatches the scheduler splits, before and after synchronizing request release.

n	split under the old launcher	split under a synchronized launcher
4	0%	0%
8	20%	0%
16	100%	60%
32	100%	100%

Table 19: Padded share under four prompt-length distributions matched on mean length, so that offered tokens are equal across arms. CV is the coefficient of variation of the sampled lengths.

length distribution	CV	padded share	TFTT p50 / p95	ITL p50 / p95
fixed-1000	0.00	8.5%	30.0 / 147.5 ms	4.9 / 9.0 ms
uniform	0.58	27.4%	45.6 / 74.7 ms	5.3 / 7.0 ms
lognormal	0.91	33.3%	43.6 / 253.3 ms	5.1 / 10.2 ms
bimodal	1.51	28.0%	17.1 / 114.2 ms	4.6 / 7.2 ms

10.10 What causes the boot cliff

That figure required measuring the cliff rather than assuming it. Configuring twenty-one shapes triggers an out-of-memory error (`RESOURCE_EXHAUSTED`) during warmup at memory fractions of 0.92, 0.90 and 0.88, and requires a reduction to 0.85 before the server starts. At 0.92 the allocation requests 32.50 MB against 12.40 MB free, and the failure is byte-identical across two independently provisioned hosts. A shortfall of 20 MB might suggest that a small reduction would suffice; it does not, since the requirement persists through three successive reductions.

The mechanism is not steady-state competition between executables and cache, which the identical capacities rule out. Every failed boot records a key-value cache size before terminating — 358,144 tokens at 0.90, 348,928 at 0.88 — so the cache is sized to fill the fraction first, and compilation then requests scratch memory against what remains. Shape coverage is charged to transient compilation headroom that the sizing step does not reserve, which is why the cost appears as a boot cliff rather than as a smaller cache.

10.11 Applying the offline procedure under traffic drift

The procedure selects a ladder from a length distribution, and a deployment’s distribution moves. Re-selection is cheap to decide and expensive to apply: the decision is arithmetic over a sample of recent lengths, while applying it requires a restart and a cold compile, measured here at 285 s for ten shapes. Re-selection is therefore a daily or weekly action rather than a continuous control loop.

How much drift a stale ladder tolerates before re-selection matters is answerable offline, without hardware, since the object under test is the selection model itself rather than a live server: fit a ladder once against a sampling-window workload (median 1200, $\sigma=0.9$, matching §4.4), then evaluate that same,

now-stale ladder against deployment-window workloads whose median has drifted by a stated factor, using the same DP and the 35 μ s/padded-token constant §4.3 measured. Compared against a ladder freshly re-fit to each drifted workload:

(`scripts/o10_drift_sensitivity.py`, self-tested; a model computation, not a hardware measurement.) The stale ladder retains 93–99% of the achievable gain across a 4 $\times$ range in median prompt length, which is more drift than a daily or weekly reselection cadence should ever accumulate against realistic diurnal prompt-length patterns. This does not validate the 35 μ s/token constant at a drifted workload’s shapes — only hardware measurement could — but it says the *selection procedure* degrades gracefully, which is the half of the review question this paper can answer without a slice.

The signal that should trigger re-selection anyway is already available to the server and costs nothing to maintain. A server knows both the compiled shape it selected for a step and the real token count in that step, so a running counter of their difference gives the realized padded share exactly, without sampling. Re-selection is warranted when that realized share exceeds what the offline model predicts for the ladder in use by more than the margin that would justify a restart. We note that `iteration_tokens_total` is not a substitute for this counter: Appendix §10.1 shows its power-of-two bins are coarser than the ladder spacing being compared. We have not built this controller, and report it as the natural operational form of the result rather than as a contribution.

One caveat bounds the model’s reach. Predicting padding requires knowing the ladder a gap will produce, and the rule is not the obvious one: the stack continues doubling while the doubling step is no larger than the gap, then switches to linear spacing. At gap 256 this inserts an entry at 768, which a reading of "powers of two, then linear" does not predict. The predictions above were computed from the corrected rule, and each arm re-reads the ladder its server printed.

Table 20: A ladder fit once, evaluated stale against workloads whose median has drifted, against one freshly re-fit to each.

deployment	median drift	stale ladder’s gain	fresh ladder’s gain	% of fresh retained
0.6× (720)		9.9 ms	10.6 ms	94%
1.0× (1200, no drift)		17.9 ms	18.2 ms	98%
1.7× (2040)		27.9 ms	29.1 ms	96%
4.0× (4800)		39.9 ms	42.9 ms	93%

Two qualifications on the BucketServe-objective comparison (§4.4). At three replays the same comparison gave $[-5.39, +1.83]$ and did not resolve, and the arms were an order of magnitude noisier; the effect is real but small enough to need the replays. And the measured gap is about half the 2.8 ms the linear model predicts from 80 padded tokens, implying roughly 20 μ s per token here against the 34.9–35.3 μ s of §4.3. The per-token cost is therefore not constant across the range: the model is accurate where padding is large, which is where ladder design is decided, and overestimates the marginal value of removing padding once little remains.

10.12 A second TPU generation: v6e-4 replicates the token-padding direction

Everything in §4 was measured on one v5litepod-4 slice, which leaves open whether the token-padding finding is a property of the mechanism — XLA’s ladder, the ragged-tile kernel RPA validates — or an artifact of one chip generation. This is answerable directly rather than argued analytically, the way §10.4 handles TP and model scale: the same stack (vLLM 0.25.0, tpu-inference 0.25.0, Qwen3-4B, TP=4) provisioned on a v6e-4 spot podslice instead of v5e, same D2/D3 ladder mechanism reported at boot (Prepared token paddings, Prepared attn request paddings: [256]), a live completion request confirmed correct output before any measurement was taken.

At $n=1$, the same three boundaries §4.2 establishes on v5e replicate in direction: 55.4%, 60.4% and 74.5% of nominal padding paid at 512/1024, 1024/2048 and 2048/4096, against v5e’s single-boundary $\sim 85\%$ figure at $n \leq 2$. The fourth boundary, 4096/8192, could not be tested: this boot’s `max_model_len` was 4096 rather than the 8192 its own config declared — a configuration-forwarding gap caught after the fact, because at the time no check in this pipeline verified a config against the live server it actually produced, only against itself — so a 4104-token prompt exceeds the context window outright, a boot-configuration fact rather than a padding measurement, and it does not affect the other three boundaries. §2’s executed-configuration check was written in response to this run and would have aborted it before any measurement was taken; this is the one instrument-definition failure in §6’s catalog that now has a guardrail, and it got one only because a replication

tripped over it in public. At $n=4$, the paid share falls to 6.9% at the 512/1024 boundary, the same batch-size pattern §4.2 reports on v5e (23.1% at $n=4$, falling toward zero by $n=16$); at $n=8$, every dispatch split across multiple prefill steps under chunked prefill and was correctly excluded rather than pooled into a misleading number, the same treatment §4.2 applies.

Two results outside §4’s four core claims came along with this boot. LENS (§10.4’s model form) fit against nine clean (bucket, n) cells with a holdout MAPE of 0.26% (worst 0.96%) — tighter than the 2.15% LENS reports on its own NPU hardware — and its fixed intercept varies by both bucket and batch size, a $7.92\times$ spread, meaning the single-threshold crossover assumed elsewhere near 2048 tokens is a special case rather than a general rule. And the step-logger instrument (§10.1) produced 1757 per-step records on this boot: padded-token overhead measured per packed step stays in a 33–59% band from $n=1$ to $n=15$ rather than falling toward zero the way the per-request measure does — the reconciliation §10.1 proposes and, on v5e’s own data, could not test.

This is one boot on one v6e-4 slice, not a second full sweep: it establishes direction, not a second interval, and the model-scale and multi-host questions §7 raises remain exactly as open on this generation as on the first.

10.13 What the GPU position statistic’s reference point actually is

§4.5 reports a position-of- $n=9$ statistic on the L4 and the abstract reports a 67 μ s / 4.0% figure from the same arms. Read carelessly these look like two normalizations of one quantity, and 16.0% looks like the same order as §4.2’s 23.1% on the token dimension — the number this paper calls the surviving exception. They are not the same quantity, and the difference is entirely in the denominator.

The statistic is $(c_9 - c_8) / (c_{16} - c_8)$. Its numerator is what a ninth request adds. Its denominator is what eight more requests add, and those eight are *real*: they carry real tokens, real key–value state and real arithmetic. A batch whose padding costs nothing, and whose cost is otherwise linear in real work, therefore does not sit at 0 on this scale. It sits at one eighth:

- fully paid ($n=9$ costs what $n=16$ costs) $\rightarrow 100\%$

- padding free, cost linear in real work $\rightarrow$ **12.5%**
- $n=9$ costs exactly what $n=8$ costs $\rightarrow$ 0%, which would mean the ninth request's *own real work* were also free, and is not the free hypothesis

Graphs measures 16.0% [15.0%, 17.1%], which is 3.5 points above the free reference rather than 16 above it. Differencing the two arms' increments instead — $(0.283 - 0.215) / 1.693$ — removes the shared real-work term directly and gives 4.0%. The two estimators agree to within half a point, which is the check that the reading above is right rather than a rationalization.

§4.2's paid share is a ratio whose denominator is nominal padding alone, with no real-work term in it. So 23.1% there and 16.0% here are not comparable magnitudes, and the cross-architecture claim does not rest on their similarity — it rests on 4.0% and 3.5 points being small.

`scripts/g1_analyze_repeats.py` reported this statistic under a docstring that called 0% "free". That gloss is wrong for the reason above and has been corrected in the script; no reported number changes, since every GPU figure in the paper was computed from the formula rather than the gloss. It is, though, exactly the instrument-definition failure §6 names as the class no guardrail covers: the estimator was right and its stated meaning was not, which no recomputation would have caught.

10.14 The BucketServe-objective comparison, with its provenance stated

§4.4 reports the direction of this comparison and sends the margin here, because the margin is smaller than the noise floor its own provenance would have to clear.

Neither ladder is expressible through `VLLM_TPU_BUCKET_PADDING_GAP`, whose family is "double while the doubling step is no larger than the gap, then step linearly". Both were compiled by patching `get_token_paddings` to accept an explicit ladder, which leaves the function unchanged when the variable is unset. Both carry fixed entries at 16, 32, 64 and 128: BucketServe's boundaries derive from prompt lengths and begin at 208, and a ladder without small entries pads a two-token decode step to 208. Both arms therefore spend fourteen shapes and differ only in where the ten free entries sit.

At equal shape count, replaying Table 6's sampled lengths: the stock ladder (10 shapes) pads 602 tokens/req for a mean end-to-end of 226.2 ms; gap-1024 pads 389 tokens/req for 215.6 ms; BucketServe's relative-waste objective pads 328 tokens/req for 209.0 ms; the padded-token objective pads 248 tokens/req for 207.4 ms. The padded-token objective is faster by 1.61 ms, 95% interval [1.04, 2.17], $p < 0.001$ over nine

replays per arm; both objectives beat the stock ladder by 17.2 and 18.8 ms.

Why that 1.61 ms is not quoted in the abstract.

The stock and gap-1024 rows here read 6.4 and 2.9 ms above Table 6's own values for nominally the same arms. That is inside the 4–11 ms between-instance offset Table 9 measures directly, and it means these two tables were not produced by one boot. Whether the four rows *within* this comparison were measured together — which is what a 1.61 ms difference with a [1.04, 2.17] interval would require to mean what it says — is not established by the record this paper has: the run is not in `captured/`, and `scripts/paper_numbers.py` does not regenerate it, so it is outside the provenance contract §2 describes. A within-boot interval an order of magnitude tighter than the between-boot floor is exactly the shape of a number that has not been asked the right question. The comparison's direction survives this, since it agrees with the cost model; the margin should not be relied on until the four arms are re-run interleaved in a single boot with the placebo cell re-measured, which is the experiment this section is short of.

Two further qualifications, and Appendix §10.11 has both in full: at three replays the same comparison gave [-5.39, +1.83] and did not resolve, and the measured gap is about half what the linear model predicts from 80 padded tokens.